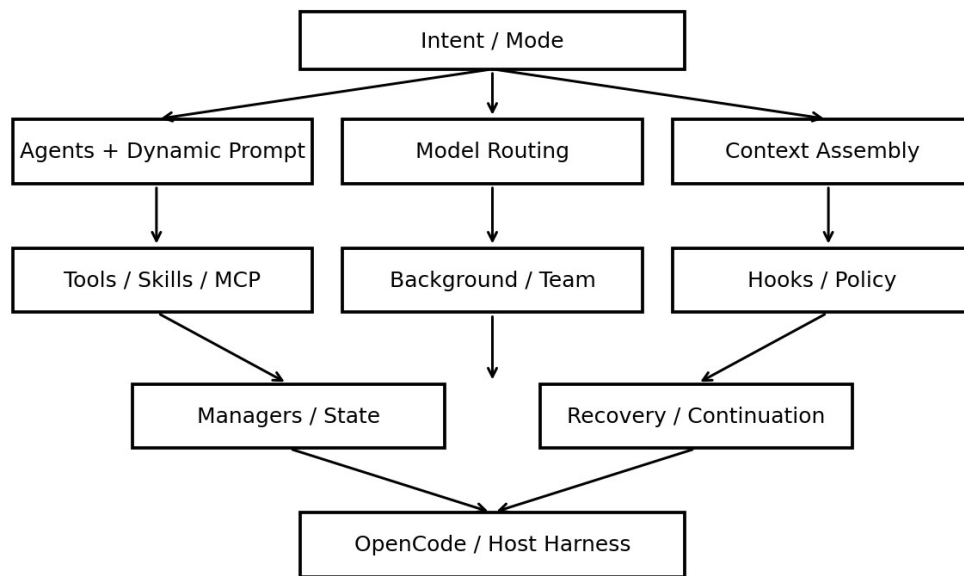


oh-my-openagent Cookbook

从安装、编排到源码级 Agent Runtime 工程实践

2026-08-13 • dev 分支源码快照 • v5.0.0-beta.7 时代

OmO Runtime Architecture



定位：这不是功能列表，而是一份“能照着做”的 Cookbook。每个 Recipe 尽量同时回答 know-how（怎么做）与 know-why（为什么这样设计）。

主要依据：官方 GitHub 仓库 dev 分支 README、installation/configuration/orchestration/features 文档，以及 packages/omo-opencode/src 下的启动、Agent、Tool、Hook、Background、Skill、Team Mode 源码。由于项目处于 multi-harness 重构期，文件与默认模型可能快速变化。

目录与阅读路线

- Part I 快速落地：安装、Doctor、统一配置
- Part II 系统心智模型：Runtime 而非 Prompt 集合
- Part III Agent 与编排：Sisyphus / Prometheus / Atlas / Specialists
- Part IV Task 与 Background Runtime
- Part V Context / Skill / MCP
- Part VI Hooks：Agent Control Plane
- Part VII Model Routing / Fallback / Permissions
- Part VIII Tool Registry 与 Hashline Edit
- Part IX Team Mode：并行多 Agent 协调
- Part X 故障诊断、迁移与生产化
- Part XI 源码阅读实验室
- Part XII 从最小 Agent Loop 演进到 Mini-OmO
- Appendix 配置模板、速查表、源码索引

READING: 推荐第一次阅读顺序：Part I → II → III → IV → VI → XI。想自己实现 Agent Runtime：直接重点读 IV、VI、VII、VIII、XII。

Part I 快速落地：安装、Doctor、统一配置

本部分目标不是“把包装上”，而是建立一个可验证、可迁移、可逐步增强的基础环境。当前 OmO 有 Ultimate(OpenCode)、Light(Codex CLI) 与 Senpi Native(beta) 三种 edition。

Edition	宿主	核心特点	适合
Ultimate	OpenCode	11 Agent、54+ lifecycle hooks、MCP、Team Mode、hashline、完整编排	学习源码与完整体验
Light	Codex CLI	移植可复用规则/工具/skill/MCP，使用 Codex 原生多 Agent 表面	已深度使用 Codex
Senpi	Standalone beta	omo 命令，Senpi engine + OMO extension	想要独立 host 的实验用户

Recipe 1. 选择 Edition

目标：根据宿主与研究目标选择 **Ultimate / Light / Senpi**。

Why：OmO 正在将共享逻辑抽成 core packages；不同 edition 并非简单“功能多少”，而是不同 harness adapter。源码学习最完整仍应从 Ultimate/OpenCode adapter 开始。

操作步骤

1. 若你要研究完整 Agent Runtime，选择 Ultimate。
2. 若你主要工作在 Codex CLI，只需要 portable capabilities，选择 Light。
3. 若你研究 multi-harness / standalone Agent OS，再额外观察 Senpi。

源码定位：README.md；docs/guide/installation.md

Recipe 2. 安装 Ultimate (OpenCode)

目标：安装完整 OmO 并让 installer 完成 provider/model 配置。

Why：Ultimate 的初始化不仅写 plugin entry，还需要把 Agent/model/provider 状态协调起来。官方推荐通过 bunx 调用 installer，而不是全局安装。

操作步骤

4. 确认 OpenCode 与 Bun 可用。
5. 运行官方 installer。
6. 按 TUI 选择 provider / subscription。
7. 完成后启动 OpenCode 并检查 Agent 列表。
8. 运行 doctor 做第二次验证。

示例

```
bunx oh-my-openagent install

# 安装后检查
bunx oh-my-openagent doctor
```

关键不变量

- 不要把“安装成功”定义为命令退出码 0；要验证 plugin、config、models、tools。

- 不要把旧版配置路径当主配置。

常见失败与修复

- 使用全局 bun/npm 安装：回到官方支持的 bunx installer 路径。
- 只看到 OpenCode 默认 Agent：先 doctor，再检查 plugin entry 与统一配置迁移。

验证

- doctor 关键检查项通过。
- OpenCode 中可见 Sisyphus/Hephaestus/Prometheus/Atlas 等核心 Agent。

源码定位：docs/guide/installation.md

Recipe 3. 安装 Light (Codex CLI)

目标：给 Codex CLI 安装 OmO Light portable components。

Why：Light 不复制 OpenCode 的 Agent registry；它把适合 Codex plugin system 的规则、MCP、Skill、continuation 等能力落到 Codex 自己的配置与插件目录。

操作步骤

9. 准备 Node/npm。
10. 运行 lazycodex-ai installer。
11. 根据自己的权限要求决定是否采用 autonomous 配置；共享环境或重要代码库建议更保守。
12. 重启 Codex session，使 hooks/plugin 状态生效。

示例

```
npx lazycodex-ai install
# 非交互安装按官方选项显式选择权限模式
```

源码定位：docs/guide/installation.md

Recipe 4. 建立统一 omo.jsonc

目标：用当前统一配置体系替代旧教程中的 oh-my-* 配置文件。

Why：2026-07 后配置统一到 ~/.omo/omo.jsonc 与项目 .omo/omo.jsonc；legacy 文件仅由 migration engine 导入。统一配置让 OpenCode/Senpi/Codex 共享基础配置，再叠加 harness/profile 层。

操作步骤

13. 创建用户层 ~/.omo/omo.jsonc。
14. 项目特定覆盖放在 <project>/.omo/omo.jsonc。
15. OpenCode 特定设置放入 "[opencode]"。
16. 需要 profile 时使用 profiles.<name>，通过 OMO_PROFILE 等激活。
17. 旧配置先 dry-run migration。

示例

```
{
  "$schema": "https://raw.githubusercontent.com/code-yeongyu/oh-my-openagent/dev/assets/omo.schema.json",
  "[opencode]": {
    "tmux": { "enabled": false },
```

```
"experimental": { "task_system": true }
}
}
```

关键不变量

- 用户层是最低优先级；更近的项目 .omo/omo.jsonc 覆盖更远祖先与用户层。
- mcp_env_allowlist 等安全敏感配置只能由用户层扩展。
- legacy 配置冲突采用 no-clobber：目标已有值优先。

源码定位：docs/reference/configuration.md

Recipe 5. 配置迁移而不破坏现有环境

目标：安全地把旧 oh-my-opencode / oh-my-openagent 配置迁移到统一配置。

Why：迁移是一次性的 lock-and-journal 流程，支持备份、冲突诊断与中断恢复。生产环境先 dry-run 比直接手工复制更可靠。

操作步骤

18. 执行 dry-run。
19. 查看冲突与目标层级。
20. 确认备份目录策略。
21. 执行真实迁移。
22. 启动 OpenCode 查看一次性 diagnostics。

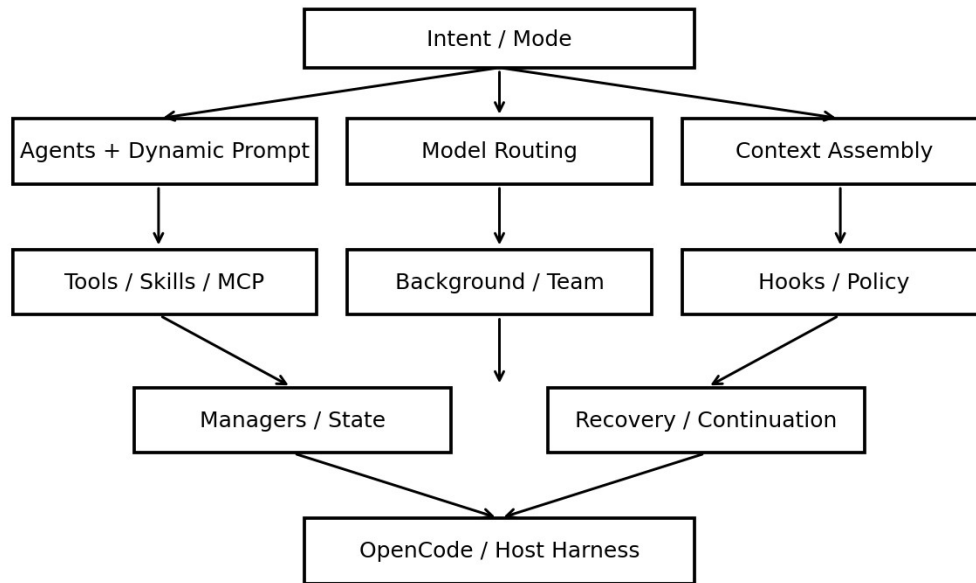
示例

```
oh-my-openagent config migrate --dry-run
oh-my-openagent config migrate
```

源码定位：docs/reference/configuration.md

Part II 系统心智模型：Runtime，而不是 Prompt 集合

OmO Runtime Architecture



最重要的理解：OmO 不是“11 个人格 Prompt”。它把 LLM 放在一个有状态、有生命周期、有恢复策略、有并行调度的 Runtime 中。Prompt 只是 Runtime capability/state 的一个投影。

层	负责什么	典型模块
Intent / Mode	ultrawork、team、plan 等当前工作模式	keyword detector / system transform
Cognitive	Agent identity、动态 Prompt、模型	agents/*
Capability	Tool / Skill / MCP	tool-registry、skill-loader、mcp
Orchestration	task、background、team	delegate-task、BackgroundManager、team-mode
Control	Hook、policy、recovery、continuation	hooks/*、plugin/*
State	session/tmux/task/model cache	Managers

Recipe 6. 从 Composition Root 理解启动

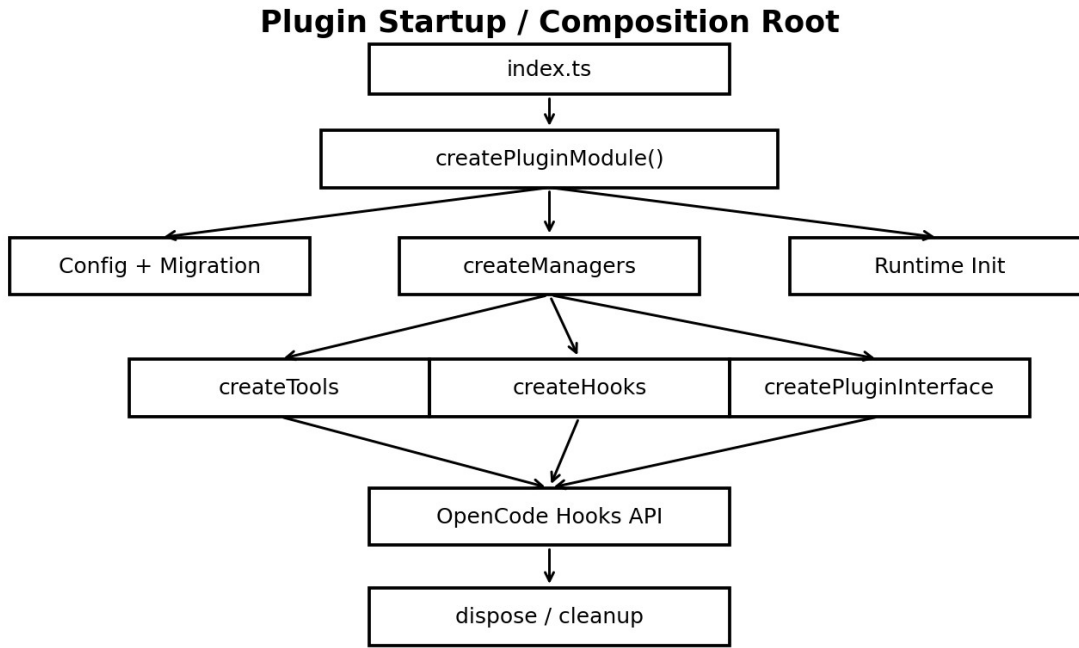
目标：沿 `createPluginModule()` 追踪所有 Runtime 组件的组装顺序。

Why：入口 `index.ts` 极薄，真正启动集中到可注入依赖的 `createPluginModule()`。这种 composition root 让副作用集中、测试可替换、host adapter 更容易抽离。

操作步骤

23. 先看 `packages/omo-opencode/src/index.ts`。
24. 再看 `testing/create-plugin-module.ts`（当前实际 composition root）。
25. 记录启动顺序：`config/migration` → `runtime init` → `managers` → `tools` → `hooks` → `plugin interface`。
26. 最后看 `dispose/cleanup`。

源码定位：`packages/omo-opencode/src/index.ts`；`testing/create-plugin-module.ts`



Recipe 7. 用 Managers / Tools / Hooks 三分法读代码

目标：快速判断一个功能应该放在哪个层。

Why: Manager 拥有长生命周期 state/resource; Tool 是给 LLM 的调用 surface; Hook 是横切 policy。把三者混在一起会导致测试困难、状态泄漏、工具变成“巨型对象”。

操作步骤

27. 看到新功能先问：它有没有跨调用状态？有则优先 Manager。
28. LLM 是否需要直接触发它？需要则提供 Tool facade。
29. 是否属于所有工具/会话都要遵守的规则？是则用 Hook。

关键不变量

- Tool 不直接拥有全局任务表。
- Hook 不应成为业务状态的唯一存储。
- Manager 不直接决定 Prompt 文案，除非通过明确 adapter。

源码定位：create-managers.ts; create-tools.ts; create-hooks.ts

Recipe 8. 认识 Plugin Interface Adapter Boundary

目标：理解 OmO 如何连接 OpenCode hooks API。

Why: `createPluginInterface()` 把内部 managers/hooks/tools 映射到 `chat.params`、`chat.message`、`event`、`tool.execute.before/after` 等宿主 API。multi-harness 重构的关键就是把这层变薄。

操作步骤

30. 阅读 `plugin-interface.ts` 的返回对象。
31. 把每个 OpenCode event 画到内部 Hook tier。

- 32. 区分 host-specific 类型和 core 数据结构。
- 33. 自己扩展时尽量把纯逻辑放到 core/helper，adapter 只做转换。

源码定位：packages/omo-opencode/src/plugin-interface.ts

Part III Agent 与编排：从角色到执行协议

当前内置 11 个 Agent：primary = Sisyphus、Hephaestus、Prometheus、Atlas；subagent = Oracle、Librarian、Explore、Multimodal-Looker、Metis、Momus、Sisyphus-Junior。不同 Agent 的差异不只是 Prompt，也包括模型链、Tool restrictions、mode 与编排职责。

Agent	角色	Mode	关键约束/价值
Sisyphus	主编排/discipline agent	primary	动态 Prompt + delegation + continuation
Hephaestus	深度自主执行	primary	GPT-oriented deep worker
Prometheus	战略规划/访谈	primary	规划，写入受 markdown 约束
Atlas	计划执行 conductor	primary	读计划、委派、验证、积累 wisdom
Oracle	架构/咨询	subagent	read-only，不写/不编辑/不继续委派
Librarian	外部 docs/OSS 研究	subagent	read-only research
Explore	代码库快速探索	subagent	read-only grep/context
Multimodal-Looker	图像/PDF	subagent	极窄工具 allowlist
Metis	规划 gap analyzer	subagent	找隐含需求与缺口
Momus	计划 reviewer	subagent	审核可执行性
Sisyphus-Junior	Category worker	subagent	聚焦执行，避免递归委派

Recipe 9. 简单任务：直接 Prompt

目标：避免无意义的编排开销。

Why：Agent orchestration 本身有延迟与上下文成本。单文件、小修复、明确问题优先直接执行。

操作步骤

- 任务是否单一、边界明确、无需并行 research？若是，直接 prompt。
- 要求最小验证（例如 diagnostics/test）。
- 只有出现不确定性/广泛搜索才委派。

源码定位：docs/guide/orchestration.md

Recipe 10. 复杂但不想详细规划：ulw / ultrawork

目标：激活更强的自主编排与上下文策略。

Why：ultrawork 是 runtime mode，不只是“更努力”的文本；keyword detector 在 transform 阶段识别 mode 并注入对应 instructions。

操作步骤

- 在任务中明确包含 ulw 或 ultrawork。
- 让 Sisyphus 先探索再执行。
- 观察是否出现 Explore/Librarian/Oracle 等委派。
- 若配置了 agents.sisyphus.ultrawork，可为该 mode 使用独立模型/reasoning。

示例

ulw

分析当前认证模块，找出 session 失效的根因，修复并跑回归测试。

源码定位：docs/guide/orchestration.md；hooks/keyword-detector

Recipe 11. 复杂且必须可审计：@plan → /start-work

目标：把 **planning context** 与 **execution context** 分离。

Why: Prometheus 负责访谈、research、plan；Metis 做 gap analysis；高准确度时 Momus + Oracle 独立 review；Atlas 再按 plan 委派执行。分离可以减少认知漂移，并让计划成为可审查的中间产物。

操作步骤

41. 用 @plan 描述目标与约束。
42. 允许 Prometheus 询问关键需求。
43. 让 Metis 检查遗漏。
44. 需要高准确时执行 dual review。
45. 计划稳定后运行 /start-work。
46. Atlas 读取计划，逐任务 delegate + verify。

关键不变量

- Planner 与 executor 的权限不同。
- 计划是中间 artifact，不应在执行阶段随意漂移。
- 每个 worker 任务必须具备可验证交付物。

源码定位：docs/guide/orchestration.md

Recipe 12. 用 Category 而不是硬编码模型

目标：把“任务意图”与“具体模型”解耦。

Why: Category 表达 workload semantics，如 quick/deep/visual-engineering/writing；Runtime 再解析 model chain。这样模型更换时不用改所有委派 Prompt。

操作步骤

47. 根据任务性质选 category。
48. 必要时 load_skills。
49. 项目专用工作负载可以自定义 category。
50. 不要在任务文本里依赖具体 model persona。

示例

```
task({
  category: "deep",
  load_skills: [],
  prompt: "定位并修复一个复杂并发问题；给出测试证据"
})
```

源码定位：docs/guide/orchestration.md；tools/delegate-task

Recipe 13. 直接调用 Specialist

目标：让特定 Agent 承担其最擅长的咨询/探索任务。

Why: 当任务“类型”不是普通执行，而明确需要架构咨询、repo grep、外部文档时，subagent_type 比 category 更精确。

操作步骤

51. 架构/复杂 tradeoff 用 oracle。
52. 代码库搜索与模式定位用 explore。
53. 官方文档、OSS 实现与外部资料用 librarian。
54. 视觉/PDF 用 multimodal-looker。

示例

```
task(subagent_type="oracle", load_skills=[], prompt="评审该并发架构", run_in_background=false)
```

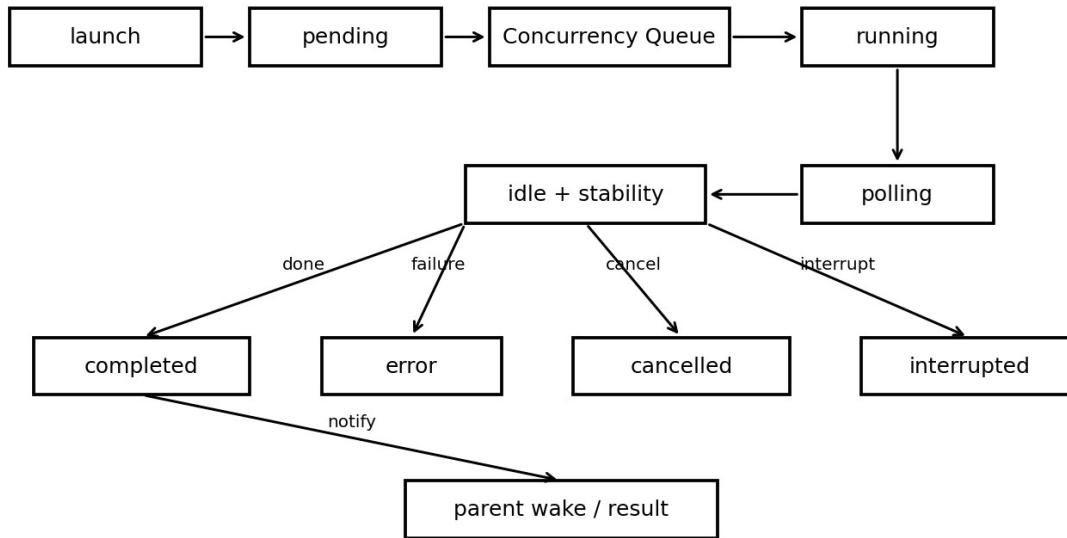
关键不变量

- category 与 subagent_type 在同一次调用中互斥。
- read-only specialist 不应被当代码执行 worker。

源码定位: docs/guide/orchestration.md; docs/reference/features.md

Part IV Task 与 Background Runtime

Background Task Lifecycle



这里是从“普通 Agent Loop”跨到 Production Multi-Agent Runtime 的分界线。delegate-task 负责解析 category/agent/model/skill，并在 sync 与 background 两条执行链之间路由；BackgroundManager 管理真正的异步生命周期。

Recipe 14. 同步 task：需要结果后才能继续

目标：建立有顺序依赖的 subagent 调用。

Why：同步执行适合“下一步必须基于专家答案”的场景。内部链条拆成 session create → prompt send → poll → result fetch，便于单独重试和测试。

操作步骤

55. 设置 run_in_background=false。
56. 给 worker 一个单目标、单交付物。
57. 拿结果后再进入主 Agent 下一步。
58. 不要用同步 task 承载大量互不依赖的 research。

源码定位：tools/delegate-task/AGENTS.md

Recipe 15. Background task：并行 research

目标：在主 Agent 继续工作时启动多个独立 subagent。

Why：并行的价值不是“Agent 越多越好”，而是隐藏等待时间并让不同搜索空间同时展开。BackgroundManager 管 pending/running/polling/completed 状态与 parent wake。

操作步骤

59. 识别互不依赖的 research 子问题。

60. 分别 `run_in_background=true`。
61. 主线程继续读代码/实现。
62. 等待 `completion notification`。
63. 需要完整输出时用 `background_output(task_id)`。

示例

```
task(subagent_type="explore", load_skills=[], prompt="查找所有 token refresh 路径", run_in_background=true)
task(subagent_type="librarian", load_skills=[], prompt="查官方 OAuth refresh best practices", run_in_background=true)
```

关键不变量

- 并行任务应彼此独立。
- 不要重复让多个昂贵模型搜索同一问题而无差异目标。

源码定位：features/background-agent/AGENTS.md；docs/reference/features.md

Recipe 16. 配置 provider/model 并发

目标：避免 background agents 把 provider rate limit 打爆。

Why：真正的限制通常是 provider/model quota，因此 OmO 支持按 provider 和 model 控制并发，而不是只有全局 semaphore。

操作步骤

64. 先按 provider 设置保守上限。
65. 对昂贵/稀缺模型再设置 modelConcurrency。
66. 观察 429/timeout 后再调。
67. cheap research model 可以更高，premium reasoning model 更低。

示例

```
"background_task": {
  "providerConcurrency": { "anthropic": 3, "openai": 3 },
  "modelConcurrency": { "anthropic/claude-opus-5": 2 }
}
```

源码定位：docs/reference/configuration.md；features/background-agent

Recipe 17. 理解“完成”为什么不是一次 idle

目标：避免 background task 在短暂停顿时被误判完成。

Why：Runtime 同时使用 session idle 与消息稳定性等信号，防止模型在工具间隙短暂 idle 被提前收割。完成判定应该是 Runtime policy，而不是只相信 LLM 的自然语言 “done”。

操作步骤

68. 记录 session.idle。
69. 结合 poll 的稳定检测。
70. 只有多信号一致才转 completed。
71. 对超时/异常使用独立 error/cancel 路径。

关键不变量

- Idle $\neq$ completed。
- LLM 说完成 $\neq$ Runtime 验证完成。

源码定位：features/background-agent/AGENTS.md

Recipe 18. Parent Wake：异步结果如何回到主会话

目标：在 background task 完成后可靠通知 parent，而不是要求主 Agent 不停轮询。

Why：parent wake 将“等待”从 LLM 上下文中的主动 polling 变成 Runtime event，节省 token 并降低上下文噪声。

操作步骤

72. background manager 完成任务。
73. result handler 生成父会话可消费的通知。
74. parent session 被唤醒/注入系统消息。
75. 主 Agent 再决定是否获取详细结果。

源码定位：features/background-agent/parent-wake-notifier.ts 等

Recipe 19. Tmux 可视化 Background Agent

目标：让多个子 Agent 的 session 以独立 pane 可观察。

Why：可视化不是编排本身，但对调试长任务、观察卡死和研究 agent behavior 很有价值。TmuxSessionManager 作为 Manager 管 session 生命周期。

操作步骤

76. 在 [opencode] 中启用 tmux.enabled。
77. 选择 layout。
78. 从 tmux 环境运行 OpenCode。
79. 观察 background session 创建与自动清理。

示例

```
"tmux": {
  "enabled": true,
  "layout": "main-vertical"
}
```

源码定位：docs/reference/features.md；create-managers.ts

Part V Context / Skill / MCP：把上下文当稀缺工作内存

OmO 的 Context Engineering 核心不是“塞更多资料”，而是根据任务、目录、Skill、Agent、Mode 动态组装最相关信息。

Recipe 20. 分层 AGENTS.md / README Context

目标：让 Agent 只获得当前目录相关规则与约定。

Why：大仓库用一个巨型全局指令文件会造成 token 浪费和规则冲突。目录注入 Hook 可以根据正在操作的路径加入局部 AGENTS.md/README。

操作步骤

80. 根目录放全局约束。
81. 子模块放局部 AGENTS.md。
82. 把只对某语言/组件有效的规则放最近目录。
83. 避免重复复制全局规则。

源码定位：hooks/directory-agents-injector；context-injector

Recipe 21. Skill Progressive Disclosure

目标：只在需要时加载领域能力。

Why：Skill 先以 name/description 暴露；Agent 决定相关后再加载完整 instructions、tools、MCP。这样能力库可以很大，而每次上下文保持小。

操作步骤

84. 把可复用领域流程写成 SKILL.md。
85. name/description 要能让 Agent 正确匹配。
86. 正文写操作协议与验证方式。
87. 只有必要时声明工具/MCP。

示例

```
---
name: frontend-review
description: Review frontend changes for accessibility and responsive behavior
tools: [read]
---

Follow the project UI conventions...
```

源码定位：features/opencode-skill-loader/AGENTS.md

Recipe 22. Skill 四级优先级

目标：用项目级 Skill 覆盖用户/内置版本，而无需 fork。

Why：同名 Skill 按 scope 去重，让组织/项目定制与 builtin 共存。

操作步骤

88. 项目 .opencode/skills 优先放项目专用版本。

- 89. OpenCode config scope 放机器级能力。
- 90. 用户 scope 放个人通用能力。
- 91. Builtin 作为最低优先级默认。

关键不变量

- 同名高优先级覆盖低优先级。
- 不要通过复制整个 builtin 目录来改一个 Skill。

源码定位: features/opencode-skill-loader/AGENTS.md

Recipe 23. Skill-embedded MCP

目标: 让 Skill 自带专用工具服务器, 并按使用时机启动。

Why: 把 MCP 与 Skill 绑定, 可以避免所有 MCP 永久出现在 Tool Schema 中; session/skill/server isolation 还能降低状态串扰。

操作步骤

- 92. 在 SKILL.md frontmatter 声明 mcp。
- 93. Skill 被选中后由 SkillMcpManager 启动/连接。
- 94. 结束时按生命周期清理。
- 95. 对于昂贵 MCP, 避免全局常驻。

源码定位: features/opencode-skill-loader; SkillMcpManager

Recipe 24. Context Compaction 后 Rehydrate State

目标: 长会话压缩后恢复目标、Todo、项目规则和必要 Runtime 状态。

Why: 只把历史消息 summary 化会丢失执行状态。OmO 用 compaction context/todo hooks 在压缩后重新注入关键状态。

操作步骤

- 96. compaction 前持久化结构化 Todo/goal。
- 97. 压缩历史。
- 98. 压缩后重新注入项目 context。
- 99. 恢复未完成 Todo。
- 100. 继续执行而不是把 summary 当完整状态。

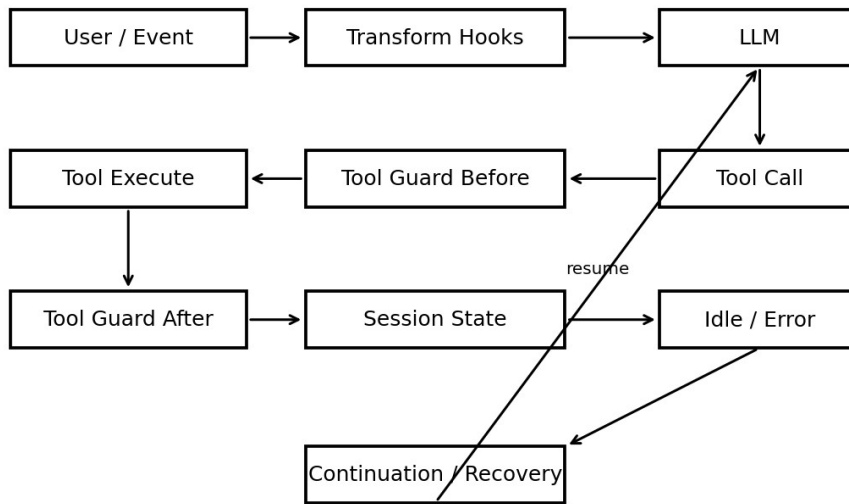
关键不变量

- Context $\neq$ State。
- 不可恢复的重要状态必须结构化持久化。

源码定位: hooks/compaction-context-injector; compaction-todo-preserver

Part VI Hooks : Agent Control Plane

Hook Pipeline: Control Plane Around the LLM



Hook 的价值是把“希望模型遵守”升级成“Runtime 可以检查、阻止、恢复”。当前系统按 Session、Tool Guard、Transform、Continuation、Skill 等层组合。

Tier	触发点	典型职责
Session	session/chat 生命周期	fallback、context limit recovery、notifications
Tool Guard	tool before/after	权限、Read-before-Write、输出截断、JSON 修复
Transform	messages/system transform	mode/context/team mailbox 注入
Continuation	idle/compaction/event	Todo 未完强制继续、background notification
Skill	chat message	Skill reminder、slash command

Recipe 25. 把安全约束从 Prompt 移到 Hook

目标：实现确定性的工具级 policy。

Why：“不要改代码”写在 Prompt 里只是软约束；tool.execute.before 可以直接拒绝不允许的操作。Prometheus md-only、write-existing-file-guard、team-tool-gating 都体现这一原则。

操作步骤

101. 定义要保护的资源/工具。
102. 在 before hook 解析调用参数。
103. 不满足条件时阻止执行并给可操作错误。
104. 将规则名加入可配置 disabled_hooks（若适用）。
105. 写 hook 单元测试。

关键不变量

- Permission boundary 必须由 Runtime enforce。
- 错误提示要告诉 Agent 下一步能做什么。

源码定位: hooks/*; plugin/hooks/create-tool-guard-hooks.ts

Recipe 26. Read-before-Write

目标：阻止 Agent 对未读过的已有文件盲写。

Why: LLM 很容易根据旧上下文或猜测覆盖文件。要求先 Read 能让编辑基于最新内容，并为 hashline/stale detection 提供依据。

操作步骤

106. before Write/Edit 检查目标是否已存在。
107. 检查当前 session 是否读过该文件。
108. 未读则拒绝并要求 Read。
109. 读后允许进入编辑流程。

源码定位: hooks/write-existing-file-guard

Recipe 27. Continuation : Todo 未完成就不算结束

目标：把 completion policy 从 LLM stop_reason 上移到 Runtime。

Why: 模型会自然停止，但 Runtime 可以在 session.idle 时检查未完成 Todo/goal，然后注入 continuation reminder。Production Agent 的“坚持完成”主要来自这种 lifecycle policy。

操作步骤

110. session.idle 时检查 task/todo state。
111. 若全部完成，允许结束。
112. 若有 unfinished，构造最小 continuation context。
113. 唤醒 session。
114. 设置 loop/unstable guard 防止无穷烧预算。

关键不变量

- Stop token $\neq$ task done。
- Continuation 必须有退出条件和预算保护。

源码定位: hooks/todo-continuation-enforcer; hooks/goal; hooks/unstable-agent-babysitter

Recipe 28. Tool Output Truncation

目标：防止一次工具输出吃掉整个 context window。

Why: 工具返回经常比模型输出更大；把 context window 当数据总线会导致推理空间迅速耗尽。Tool after hook 可以截断/摘要非关键输出。

操作步骤

115. 给不同 tool 类型设置输出预算。
116. 保留头尾、错误与关键标识。

- 117. 大结果存文件/引用，而不是全部回灌。
- 118. 必要时提供后续分页/检索工具。

源码定位：hooks/tool-output-truncator

Recipe 29. Error Recovery Hook

目标：把常见结构化错误变成可恢复动作，而不是整次任务失败。

Why：编辑、JSON、provider、context limit 等失败模式具有可识别结构。Hook 可以分类错误并注入精确 correction guidance。

操作步骤

- 119. 先分类 error。
- 120. 判断能否安全重试。
- 121. 给 LLM 最少必要的错误上下文。
- 122. 限制重试次数/检测循环。
- 123. 不可恢复则升级给 parent/user。

源码定位：edit-error-recovery；json-error-recovery；runtime-fallback；anthropic-context-window-limit-recovery

Part VII Model Routing / Fallback / Permissions

模型选择是 Runtime policy。当前共享逻辑已抽到 model-core；OpenCode adapter 提供已连接 provider/model cache。

Recipe 30. Agent Model Override + Fallback Chain

目标：为 Agent 定义主模型与可恢复模型链。

Why：Provider availability 和 quota 会变化。Production Agent 应在配置层表达 model chain，而不是在 Prompt 中硬编码。

操作步骤

124. 为关键 Agent 设置 model 或 models。
125. 按任务特性配置 reasoning。
126. 给 fallback rung 设置独立 temperature/reasoning。
127. 让 Runtime 检查 provider 可用性。

示例

```
"agents": {
  "sisyphus": {
    "model": "anthropic/claude-opus-5",
    "fallback_models": [
      "openai/gpt-5.6-sol",
      { "model": "google/gemini-3.1-pro", "reasoning": "high" }
    ]
  }
}
```

源码定位：docs/reference/configuration.md；shared/model-resolution-pipeline.ts

Recipe 31. 区分 Proactive 与 Reactive Fallback

目标：请求前选可用模型；请求失败后还能切换。

Why：只做一种 fallback 不够：proactive model-fallback 在 chat.params 前避免明显不可用模型；runtime-fallback 在真实 API 错误后处理动态失败。

操作步骤

128. 请求前根据 provider cache/model capability 解析。
129. 发起请求。
130. 捕获 provider/API error。
131. 按错误类别决定是否切下一个模型。
132. 保持 task/session continuity。

源码定位：hooks/model-fallback；hooks/runtime-fallback

Recipe 32. 用权限而不是 Persona 控制 Agent

目标：明确每个 Agent 能 edit/bash/webfetch/访问外部目录的策略。

Why: Agent Prompt 可以描述职责，但权限必须由 config/builtin restrictions/OpenCode gate/hooks 等确定性机制执行。

操作步骤

133. 为 read-only Agent 显式 deny edit。
134. 危险 bash 可设置 ask 或按命令 map。
135. 只暴露需要的 Tool。
136. 用 disabled_tools/disabled_agents 做更粗粒度裁剪。

示例

```
"agents": {  
  "explore": {  
    "permission": {  
      "edit": "deny",  
      "bash": "ask",  
      "webfetch": "allow"  
    }  
  }  
}
```

关键不变量

- AGENTS.md 是 instruction context，不是 deterministic permission boundary。

源码定位: docs/reference/configuration.md; docs/reference/features.md

Part VIII Tool Registry 与 Hashline Edit

OmO 的 Tool Surface 是按环境和 feature gate 动态构建的，而不是固定几十个 schema 永久塞给模型。

Recipe 33. 动态 Tool Registry

目标：根据配置、环境、Team/Monitor/Task feature gate 构造实际 Tool Surface。

Why：工具越多，模型选择错误和 schema token 都越多。Registry 先组合 core/gated tools，再 normalize schema、filter disabled、按 max_tools 裁剪。

操作步骤

137. 建立 core tools。
138. 根据 interactive bash/team/monitor/task/hashline feature 条件加工具。
139. 统一 normalize argument schema。
140. 应用 disabled_tools。
141. 必要时 max_tools trimming。

源码定位：plugin/tool-registry.ts

Recipe 34. Hashline Edit：给文件编辑加乐观并发控制

目标：避免 Agent 使用 stale line/reference 覆盖已变化的文件。

Why：LINE#HASH 把“我读到的是哪一版这行”编码进编辑引用。执行时重新计算 hash，不匹配就拒绝，等价于数据库 optimistic concurrency control。

操作步骤

142. Read 输出每行 LINE#ID。
143. Agent 用该 anchor 构造 replace/append/prepend。
144. 执行前重新校验 anchor hash。
145. 多编辑按 bottom-up 排序。
146. 成功后输出 diff；冲突时要求 re-read。

示例

```
# read
42#VK| function hello() {

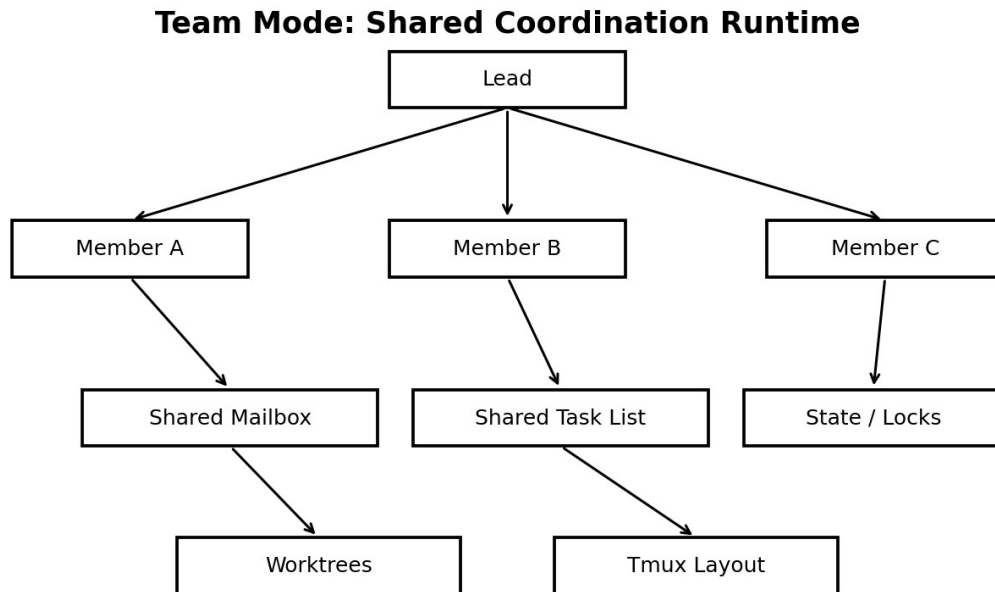
# edit
{ "op": "replace", "pos": "42#VK", "lines": "function hello(name: string) {" }
```

关键不变量

- stale hash 必须 reject。
- 多编辑 bottom-up 以避免前面插删改变后续行号。

源码定位：tools/hashline-edit/AGENTS.md

Part IX Team Mode：并行多 Agent 协调



Team Mode 与 background delegation 不同：background 主要是 parent→child fan-out；Team Mode 引入 shared mailbox、task list、claim、member lifecycle、state/locks、worktrees 和可选 tmux layout，更像小型分布式协调系统。

Recipe 35. 何用 Team Mode

目标：只在真正需要长期成员协作与共享任务队列时使用 Team Mode。

Why：普通并行搜索使用 background task 更轻。只有需要 peer communication、shared claiming、长时间 member 生命周期时，Team Mode 的复杂度才值得。

操作步骤

147. 先确认 background delegation 不足。
148. 启用 team_mode.enabled。
149. 设置 max_parallel_members/max_members/时间与消息上限。
150. 配置可选 tmux_visualization。
151. 用 TeamSpec 创建明确角色。

示例

```

"team_mode": {
  "enabled": true,
  "max_parallel_members": 4,
  "max_members": 8,
  "max_wall_clock_minutes": 120
}
  
```

源码定位：features/team-mode/AGENTS.md

Recipe 36. Mailbox + Deferred Ack

目标：让 team members 异步传递消息而不把每次发送变同步 RPC。

Why：Deferred ack 降低耦合；消息先入 mailbox，recipient 独立 poll/ack。Transform hook 可以把待处理 mailbox 信息注入上下文。

操作步骤

152. lead/member 使用 team_send_message。
153. 消息写入 recipient mailbox。
154. recipient poll 获取未读。
155. 处理后 ack。
156. 控制单消息与未读总字节上限。

源码定位：features/team-mode/team-mailbox；team-mailbox-injector

Recipe 37. Shared Task Claim 要加原子锁

目标：防止两个成员同时认领同一任务。

Why：这是典型并发一致性问题，不能靠“大家自觉”。Team task update 使用文件锁/原子状态变更保证单一 owner。

操作步骤

157. 创建 shared task。
158. member 尝试 claimed transition。
159. 在锁内检查当前状态。
160. 仅一个 claimant 成功。
161. 完成后写 completed/result。

关键不变量

- claim 必须原子。
- state 写入采用 temp + rename/等价原子模式。

源码定位：features/team-mode/team-tasklist；team-state-store

Recipe 38. 避免 Nested Teams

目标：限制 orchestrator branching factor。

Why：如果每个 member 都能创建 team，成员数量和消息拓扑会指数膨胀。team-tool-gating 明确禁止成员调用 team_create。

操作步骤

162. 只允许 lead 创建 team。
163. member 需要专家时走普通 task/delegation。
164. TeamSpec parse 时拒绝不适合的 read-only agent。

源码定位：features/team-mode/AGENTS.md；team-tool-gating

Part X 故障诊断、迁移与生产化

Recipe 39. Doctor-first 排障

目标：建立确定性的诊断入口。

Why：复杂 Agent 系统涉及 host、provider、models、hooks、tool、tmux、team 等多层。没有 doctor，用户只能靠猜。

操作步骤

165. 先 doctor。
166. 按 System/Config/Models/Tools/Team 等层定位。
167. 不要第一时间改 Prompt。
168. 修复最低层基础设施后再复测。

示例

```
bunx oh-my-openagent doctor
# 需要机器可读时使用官方支持的 JSON 选项（如当前 CLI 提供）
```

源码定位：installation docs; cli/doctor

Recipe 40. 避免 Host 初始化重入死锁

目标：插件初始化时不反向调用可能等待插件完成的 host API。

Why：createBuiltinAgents 源码明确记录了初始化期调用 OpenCode client API 曾导致 deadlock，因此改用 provider/model cache。

操作步骤

169. 识别 host startup callback。
170. 不要在回调内调用会等待完整初始化的 host service。
171. 提前构建 cache/snapshot。
172. 初始化完成后再允许正常 host API。

关键不变量

- Host waits Plugin + Plugin waits Host = deadlock。

源码定位：agents/builtin-agents.ts

Recipe 41. Process Cleanup 与资源回收

目标：确保 tmux、background、team、monitor、MCP 在 session/process 结束时 best-effort 清理。

Why：Agent Runtime 会创建子进程、pane、server、文件锁；没有统一 dispose 会导致 zombie state。

操作步骤

173. 所有长生命周期组件提供 cleanup/shutdown。
174. composition root 注册 dispose。
175. process exit 走 best-effort cleanup。
176. cleanup 本身不可阻断主退出。

源码定位: create-managers.ts; plugin-dispose.ts; background-agent/process-cleanup.ts

Recipe 42. Telemetry/日志也要遵守最小信息原则

目标：让生产诊断可用，但不要把敏感代码/Prompt 当 telemetry。

Why：官方 telemetry 设计强调有限频率与安装标识哈希。自己扩展时更应该区分 operational metric 与 user content。

操作步骤

- 177. 记录状态、耗时、错误类别。
- 178. 避免上传源代码、完整 Prompt、密钥。
- 179. 提供 opt-out。
- 180. 本地 debug 日志与远程 telemetry 分离。

源码定位: README / telemetry docs; shared/posthog

Part XI 源码阅读实验室

不要从巨型 Prompt 开始读。先建立“启动 → 状态 → 能力 → 生命周期 → 编排”的骨架，再钻入 Agent prompt。

顺序	路径	你要回答的问题
01	packages/omo-opencode/src/index.ts	入口到底做了什么？
02	testing/create-plugin-module.ts	Runtime 以什么顺序组装？
03	plugin-interface.ts	Host API 与内部 Runtime 的边界在哪？
04	create-managers.ts	哪些对象拥有长生命周期 state？
05	create-tools.ts + plugin/tool-registry.ts	Tool Surface 怎样按 feature 动态生成？
06	create-hooks.ts + plugin/hooks/*	Hook tier 如何组合？
07	agents/builtin-agents.ts	Agent registry 与 model resolution 如何连接？
08	agents/dynamic-agent-*	Prompt 如何由 Runtime capability 动态生成？
09	tools/delegate-task/*	task 如何解析 category/agent/model/skill？
10	features/background-agent/*	异步状态机、并发、poll、parent wake？
11	features/opencode-skill-loader/*	Skill 如何发现、合并、加载？
12	plugin-handlers/*	6-phase config pipeline 为什么按这个顺序？
13	shared/model-resolution-pipeline.ts	哪些逻辑已抽到 model-core？
14	tools/hashline-edit/*	编辑一致性怎样保证？
15	features/team-mode/*	Mailbox/lock/state/worktree 如何协调？

Lab 1. 画出 createPluginModule 的副作用

目标：识别所有启动时 side effect 与可测试 seam。

Why：Dependency injection 的价值只有在你能指出“哪些副作用被替换”时才真正理解。

操作步骤

181. 列出 defaultPluginModuleDeps。
182. 给每个依赖标注 pure / IO / process / network / host。
183. 找出 startup 中 fire-and-forget 的调用。
184. 检查每个 failure 是否被隔离。
185. 找 dispose 对称关系。

源码定位：testing/create-plugin-module.ts

Lab 2. 逆向一个最终 Sisyphus Prompt

目标：理解 Prompt 是 capability projection。

Why：动态 Prompt Builder 把 availableAgents、availableSkills、availableCategories、tool categorization、policy sections 组合成运行时 system prompt。

操作步骤

186. 从 dynamic-agent-prompt-builder.ts 看导出 section。
187. 追 availableAgents 从 builtin agent collection 如何产生。
188. 禁用一个 agent，观察 delegation section 应如何变化。
189. 增加一个 category，观察 category/skill guide。

源码定位：agents/dynamic-agent-*；agents/builtin-agents.ts

Lab 3. 从 task() 跟踪到 BackgroundManager

目标：把一次 LLM Tool Call 追到真正 async session。

Why：这是理解 Multi-Agent Runtime 最关键的一条调用链。

操作步骤

190. createDelegateTask() 入口。
191. category/subagent resolver。
192. model selection + skill resolver。
193. executor 判断 sync/background。
194. BackgroundManager.launch。
195. spawner 建 session。
196. poller/idle event。
197. result handler + parent wake。

源码定位：tools/delegate-task；features/background-agent

Lab 4. 做一个新 Hook 的最小改动

目标：学会 Hook factory、tier registration、config allowlist、test 四件套。

Why：源码 AGENTS.md 已给出 adding hook 规范。真正难点是“选对 tier”和“不要制造隐式顺序依赖”。

操作步骤

198. 新建 hook directory + index factory。
199. 根据 event 选择 session/tool-guard/transform/continuation/skill tier。
200. 加入 HookName schema。
201. co-located test。
202. 验证注册顺序是否影响已有 hook。

源码定位：hooks/AGENTS.md

Part XII 从最小 Agent Loop 演进到 Mini-OmO

如果你正在从 learn-claude-code s01/s02/s03 继续往前走，可以把 OmO 拆成一条渐进式实现路线。不要直接复制 50+ Hooks。

Stage	新增能力	核心抽象
s01	LLM↔Tool loop	AgentLoop
s02	多工具分发	ToolRegistry
s03	权限 gate	Policy/Permission
s04	事件与 Hook	HookPipeline
s05	结构化 state + Todo continuation	SessionState / CompletionPolicy
s06	Subagent sync delegation	TaskExecutor
s07	Background Agent	BackgroundManager + Concurrency
s08	Dynamic Context / Skill	ContextAssembler / SkillLoader
s09	Model Routing / fallback	ModelResolver
s10	Recovery / compaction	RecoveryController
s11	Hashline edit	Optimistic File Edit
s12	Team coordination (可选)	Mailbox / TaskList / Locks

Recipe 43. Mini-OmO s04：建立统一 Hook Bus

目标：把 tool/session/message 的横切规则从 Agent Loop 主体移出去。

Why：Agent Loop 应保持小；策略通过 event hooks 组合。第一版只做 5 个事件即可：message.transform、tool.before、tool.after、session.idle、session.error。

操作步骤

203. 定义 HookContext。
204. 按 event 注册有序 handler。
205. handler 可以 mutate context 或返回 block/retry signal。
206. 给 Hook 顺序写测试。

示例

```
class HookBus:
    def emit(self, event, ctx):
        for hook in self.hooks[event]:
            result = hook(ctx)
            if result.blocked:
                return result
        return ctx
```

Recipe 44. Mini-OmO s05：CompletionPolicy

目标：不再只依赖模型 stop_reason 判断任务结束。

Why：你需要 Runtime 自己检查 todo/tests/background/deadline。

操作步骤

207. 定义 structured todos。
208. session idle 时评估 completion predicate。
209. 未完成则注入 continuation。
210. 设置最大 continuation turn / budget。

示例

```
def is_complete(state):
    return all(t.done for t in state.todos) and not state.background_running
```

Recipe 45. Mini-OmO s07 : BackgroundManager

目标：实现可控的异步 subagent，而不是裸 asyncio.create_task。

Why：Production BackgroundManager 至少需要 task state、concurrency key、cancel、result history、timeout 与 notification。

操作步骤

211. 定义 BackgroundTask state enum。
212. 按 provider/model key 获取 semaphore。
213. spawn 子 session。
214. poll + event 组合检测完成。
215. finally 释放 slot。
216. notify parent。

示例

```
pending -> running -> polling -> completed
                        -> error
                        -> cancelled
```

Recipe 46. Mini-OmO s08 : ContextAssembler

目标：让最终 Prompt 来自 Runtime capability/state，而不是手工维护巨型模板。

Why：当 Agent/Tool/Skill 可被禁用或新增时，静态 Prompt 一定会过时。

操作步骤

217. 收集 available tools。
218. 收集 available agents。
219. 匹配目录规则。
220. 只加载相关 Skill。
221. 生成 delegation/tool selection/policy sections。

示例

```
system_prompt = compose(
    identity,
    env_context,
    available_agents,
    available_tools,
    matched_rules,
    loaded_skills,
    policy
)
```

Recipe 47. Mini-OmO s11 : Hash-Anchored Edit

目标：给自己的 coding agent 加 stale edit 防护。

Why：相比复杂 diff matching，一个短内容 hash + line anchor 已能显著降低并发/重复编辑冲突。

操作步骤

- 222. Read 时生成 line_hash。
- 223. Edit 参数携带 line+hash。
- 224. 写入前验证。
- 225. 冲突要求重新 Read。
- 226. 多 edit bottom-up。

Recipe 48. 什么时候不要实现 Team Mode

目标：控制系统复杂度。

Why：如果你的任务只是并行 research，BackgroundManager 足够。Team Mode 会引入持久 state、mailbox、locks、recovery、worktrees 和成员权限矩阵。

操作步骤

- 227. 先实现 parent→child delegation。
- 228. 再实现 background concurrency。
- 229. 只有出现 peer coordination 的真实需求，再上 Team Mode。

Production Agent Runtime Checklist

- ☐ 入口是否只有 composition，不塞业务逻辑？
- ☐ Tool/Manager/Hook 边界是否清晰？
- ☐ 是否存在结构化 SessionState？
- ☐ LLM stop 是否会被 CompletionPolicy 二次验证？
- ☐ Provider/model 是否有并发控制？
- ☐ 是否支持 cancel/timeout/cleanup？
- ☐ 工具输出是否有 token budget？
- ☐ Compaction 后能否 rehydrate critical state？
- ☐ Agent 权限是否由 Runtime enforce，而非只靠 Prompt？
- ☐ 动态 Prompt 是否与当前可用 Tool/Agent/Skill 一致？
- ☐ 模型失败是否有 proactive + reactive fallback？
- ☐ 文件 edit 是否检测 stale state？
- ☐ 是否有 doctor/diagnostics？
- ☐ 是否能在测试中替换 network/process/host dependencies？
- ☐ 日志/telemetry 是否避免敏感内容？

Appendix A 推荐基础配置模板

```
{
  "$schema": "https://raw.githubusercontent.com/code-yeongyu/oh-my-openagent/dev/assets/omo.schema.json",
  "[opencode]": {
    "agents": {
      "sisyphus": {
        "model": "kimi-for-coding/kimi-k3",
        "ultrawork": { "model": "anthropic/claude-opus-5", "reasoning": "max" }
      },
      "librarian": { "model": "google/gemini-3.6-flash" },
      "explore": { "model": "github-copilot/grok-code-fast-1" },
      "oracle": { "model": "openai/gpt-5.6-sol", "reasoning": "high" }
    },
    "categories": {
      "quick": { "model": "kimi-for-coding/kimi-for-coding-highspeed" },
      "writing": { "model": "kimi-for-coding/kimi-k3", "reasoning": "low" },
      "git": {
        "model": "opencode/gpt-5-nano",
        "description": "All git operations",
        "prompt_append": "Focus on atomic commits, clear messages, and safe operations."
      }
    },
    "background_task": {
      "providerConcurrency": { "anthropic": 3, "openai": 3 },
      "modelConcurrency": { "anthropic/claude-opus-5": 2 }
    },
    "experimental": { "task_system": true },
    "tmux": { "enabled": false }
  }
}
```

CONFIG: 这是学习用起点，不是“最佳模型配置”。模型/provider 可用性和价格变化快，应让 **doctor + model resolution** 验证实际环境。

Appendix B 快速决策表

场景	首选机制	避免
单文件小修	直接 Prompt	开 Team Mode
广泛代码搜索	Explore background	主 Agent 手工逐目录翻
外部文档/OSS	Librarian	把全部网页灌进主 context
架构 tradeoff	Oracle	让 read-only consultant 写代码
复杂需求但懒得规划	ulw	先写超长手工 prompt
必须精确审计	@plan → /start-work	边聊边改造成计划漂移
并行独立 research	background task	同步串行等待
长期成员协作	Team Mode	用一堆无状态 background task 模拟共享队列
大型工具输出	truncation + artifact/reference	完整回灌 context
文件频繁修改	hashline edit	只按旧 line number 编辑

Appendix C 关键源码路径速查

主题	路径
----	----

Composition root	packages/omo-opencode/src/testing/create-plugin-module.ts
OpenCode adapter	packages/omo-opencode/src/plugin-interface.ts
Managers	packages/omo-opencode/src/create-managers.ts
Tools	packages/omo-opencode/src/create-tools.ts; plugin/tool-registry.ts
Hooks	packages/omo-opencode/src/create-hooks.ts; plugin/hooks/*; hooks/*
Agents	packages/omo-opencode/src/agents/*
Dynamic prompt	packages/omo-opencode/src/agents/dynamic-agent-*
Delegation	packages/omo-opencode/src/tools/delegate-task/*
Background	packages/omo-opencode/src/features/background-agent/*
Skills	packages/omo-opencode/src/features/opencode-skill-loader/*
Config pipeline	packages/omo-opencode/src/plugin-handlers/*
Model core adapter	packages/omo-opencode/src/shared/model-resolution-pipeline.ts
Hashline	packages/omo-opencode/src/tools/hashline-edit/*
Team Mode	packages/omo-opencode/src/features/team-mode/*

Appendix D Sources & Snapshot

快照日期：2026-08-13。GitHub 默认分支：dev。检索到的 latest GitHub release：v5.0.0-beta.7（published 2026-08-12 UTC）。项目正处于 Multi-Harness Agent OS Refactor。

- <https://github.com/code-yeongyu/oh-my-openagent/blob/dev/README.md>
- <https://github.com/code-yeongyu/oh-my-openagent/blob/dev/docs/guide/installation.md>
- <https://github.com/code-yeongyu/oh-my-openagent/blob/dev/docs/reference/configuration.md>
- <https://github.com/code-yeongyu/oh-my-openagent/blob/dev/docs/guide/orchestration.md>
- <https://github.com/code-yeongyu/oh-my-openagent/blob/dev/docs/reference/features.md>
- packages/omo-opencode/src/testing/create-plugin-module.ts
- packages/omo-opencode/src/create-managers.ts
- packages/omo-opencode/src/create-tools.ts
- packages/omo-opencode/src/create-hooks.ts
- packages/omo-opencode/src/plugin-interface.ts
- packages/omo-opencode/src/plugin/tool-registry.ts
- packages/omo-opencode/src/agents/AGENTS.md
- packages/omo-opencode/src/hooks/AGENTS.md
- packages/omo-opencode/src/tools/delegate-task/AGENTS.md
- packages/omo-opencode/src/features/background-agent/AGENTS.md
- packages/omo-opencode/src/features/opencode-skill-loader/AGENTS.md
- packages/omo-opencode/src/features/team-mode/AGENTS.md
- packages/omo-opencode/src/tools/hashline-edit/AGENTS.md
- packages/omo-opencode/src/plugin-handlers/AGENTS.md

VERSION: 版本提醒：本仓库演进非常快。Cookbook 中“架构原则/不变量”通常比“默认模型名、Hook 数量、具体文件位置”更稳定。做二次开发时应优先从当前 dev 的 AGENTS.md、schema 与 adapter shim 验证。